DSA & LeetCode 150 Patterns Master Revision Guide

The definitive high-yield blueprint: 14 algorithmic patterns, time & space Big-O matrix, clean implementation templates, and essential interview mental models.

LITHAVEN CAMPUS STUDY VAULT • OFFICIAL REVISION CAPSULE
COURSE: CS201 Data Structures & Algorithms | **TARGET:** Top Tier Product Engineering & Campus Placements | **CIRCULATION:** Free Open Academic

1. The 14 Core Algorithmic Problem-Solving Patterns

Pattern	When to Use (Problem Clue)	Structure	Complexity	LeetCode Benchmarks
Two Pointers	Sorted arrays/linked lists; searching pairs or palindromes without extra space.	Array / LinkedList	`O(N) time, O(1) space`	Two Sum II, 3Sum, Container Most Water
Sliding Window	Contiguous subarrays/substrings satisfying min/max/length condition.	Array, String, Hash Map	`O(N) time, O(K) space`	Longest Substring w/o Repeat, Min Window Substring
Fast & Slow Pointers	Cycle detection in sequences or linked lists; finding middle node.	LinkedList, Array	`O(N) time, O(1) space`	LinkedList Cycle, Find Duplicate Number
Merge Intervals	Overlapping intervals, scheduling, calendar collisions, disjoint ranges.	Array of Intervals	`O(N log N) time, O(1) space`	Merge Intervals, Insert Interval, Non-overlapping
Cyclic Sort	Arrays with numbers in fixed range 1..N or 0..N finding missing/duplicated elements.	Array (In-Place)	`O(N) time, O(1) space`	Missing Number, Find All Duplicates, First Missing Positive
In-place Reversal	Reversing sublists in singly linked lists without allocating new nodes.	LinkedList	`O(N) time, O(1) space`	Reverse Linked List, Reverse Nodes in k-Group
Tree BFS (Level Order)	Level-by-level traversal, shortest path in unweighted tree, tree zigzag.	Queue (FIFO)	`O(N) time, O(W) space`	Binary Tree Level Order, Right Side View
Tree DFS (Pre/In/Post)	Root-to-leaf paths, maximum depth, tree isomorphism, subtree checking.	Call Stack / Explicit Stack	`O(N) time, O(H) space`	Max Path Sum, Lowest Common Ancestor, Validate BST
Two Heaps	Calculating rolling median or streaming percentiles on incoming stream.	Min-Heap & Max-Heap	`O(log N) insert, O(1) median`	Find Median from Data Stream, IPO
Subsets & Backtracking	Combinations, permutations, powersets, decision trees with constraint pruning.	Recursion / State Array	`O(2^N) or O(N!) time`	Subsets, Permutations, Combination Sum, Word Search
Modified Binary Search	Searching sorted or rotated sorted arrays with logarithmic guarantees.	Array (Sorted)	`O(log N) time, O(1) space`	Search in Rotated Sorted Array, Find Peak Element
Top 'K' Elements	Finding K largest/smallest elements in unsorted collection without full sort.	Min-Heap or QuickSelect	`O(N log K) time, O(K) space`	Kth Largest Element, Top K Frequent Elements
K-way Merge	Merging K sorted lists or sorted arrays into single sorted sequence.	Min-Heap of size K	`O(N log K) time, O(K) space`	Merge k Sorted Lists, Kth Smallest in Sorted Matrix
0/1 Knapsack & DP	Optimal subset choices under budget, overlapping subproblems, memoization.	DP Table / State Array	`O(N * W) time, O(W) space`	Coin Change, Partition Equal Subset, Target Sum

2. Algorithmic Big-O Complexity Master Matrix

Data Structure	Access (Avg)	Search (Avg)	Insertion	Deletion	Worst Space
Array (Dynamic)	`O(1)`	`O(N)`	`O(1) amortized`	`O(N)`	`O(N)`
Singly Linked List	`O(N)`	`O(N)`	`O(1) at head`	`O(1) given node`	`O(N)`
Hash Table / Map	`N/A`	`O(1) avg / O(N) worst`	`O(1) avg`	`O(1) avg`	`O(N)`
Binary Search Tree	`O(log N)`	`O(log N)`	`O(log N)`	`O(log N)`	`O(N) unbal.`
Red-Black / AVL Tree	`O(log N)`	`O(log N)`	`O(log N)`	`O(log N)`	`O(N) strict`
Min / Max Binary Heap	`O(1) peek`	`O(N)`	`O(log N)`	`O(log N) pop`	`O(N)`
Trie (Prefix Tree)	`O(L) word len`	`O(L) word len`	`O(L) word len`	`O(L)`	`O(Alphabet * L * N)`

3. Pro-Interview Mental Models & Pitfall Checklist

- **Off-by-One Guardrail:** When calculating midpoint, always use mid = left + (right - left) // 2 to prevent 32-bit integer overflow.
- **Dummy Head Pattern:** In LinkedList node deletion or splicing, always instantiate dummy = ListNode(0, head) to eliminate special case branches for the head node.
- **Subarray vs Subsequence:** Subarrays are strictly contiguous (Sliding Window / Prefix Sum). Subsequences allow omissions while preserving index order (DP / Greedy).
- **Space Compression:** When DP transition only depends on dp[i-1] and dp[i-2], compress the table to 2 scalar variables for O(1) auxiliary space.
- **BFS Queue Enqueue Mark:** Always mark graph nodes as visited immediately upon enqueueing, never on dequeueing, to prevent exponential queue ballooning in dense cyclic graphs.